



UNIT INFORMATION	
COURSE – (LEADING TO)	B.Sc (Hons) Software Engineering
UNIT NAME	Business Component Development I
UNIT ID	JIAT/BCD I
ASSIGNMENT ID	JIAT/BCD I/EX/01

STUDENT INFORMATION	
STUDENT NAME	Mohamed Fareed Abdullah
STUDENT ID	200506301171
BATCH	Cohort 8 - BCU
BRANCH	Kandy

2. Critical Analysis and Test Report

2.1 Testing Strategy Overview

Given the brief's emphasis on critical thinking over exhaustive implementation, testing was carried out at three levels: unit-level timing tests built on a shared `TestTimingUtils.measureMillis()` helper; JUnit-based simulations exercising latency thresholds and load-style loop counts without a running EJB/JMS/MySQL stack; and a JMeter load plan (`techmart_order_test.jmx`) executed against the actually deployed application on Payara, the only level generating real HTTP traffic against the real Servlets, EJBs and database described in the TID. Each level is evaluated below for what it does and does not prove.

2.2 Unit and Integration Testing

`TestTimingUtils.measureMillis()` wraps a `Runnable` (or a plain millisecond value) with `System.nanoTime()` calls before and after execution. Three JUnit 5 tests are built on it. `AsyncExecutionTest` times a single `println` and asserts the elapsed time stays under 50 ms, standing in for triggering the real asynchronous call without blocking the caller. `LoadSimulationTest` times a 1,000-iteration loop against a 2,000 ms threshold, standing in for a high-volume order-processing scenario. `LatencyThresholdTest` is a parameterised test that calls `Thread.sleep()` for a supplied duration and asserts it stays under a named threshold, covering Database Query Latency, MDB Processing Time and Order Processing Latency (NFR).

None of these three tests invokes `ProductServiceBean`, `OrderServiceBean`, `OrderProcessingMDB` or a real `EntityManager` query; each measures a `println`, an empty loop, or `Thread.sleep()` in place of the real operation. They confirm the timing harness and thresholds work as written, not that the deployed EJB/JMS stack behaves this way under equivalent load.

2.3 Test Coverage Gap Analysis

Three pieces of pure, deterministic logic were initially untested despite being cheap to cover with mocked data: `OrderServiceBean`'s stock-sufficiency check (summing `Inventory.quantity` per product and throwing `InsufficientStockException` when demand exceeds supply); `InsufficientStockException`'s constructor and getters, which carry `productName`, `requestedQuantity` and `availableQuantity` for the servlet's error redirect; and, most significantly given TID §1.8, `deductStockProportionally()`'s rounding arithmetic, which computes each warehouse's share, absorbs the remainder into the largest-stock warehouse, and clamps each deduction. All three are pure functions or exception classes needing no running container, JMS broker or live database, making their absence a low-cost-to-close gap. These gaps, along with several beans that had no coverage at all, were subsequently closed with eighteen additional test cases (Appendix A, Table 2).

2.4 Performance and Load Testing

techmart_order_test.jmx defines one thread group, “Concurrent Buyers”, ramped to 10,000 threads over a 10-second ramp time, each running Step 1 (“Add Product to Cart”, random productId 1-12) then Step 2 (“Confirm and Place Order”, random customerId 1-5) once - a genuine attempt at a 10,000-user checkout run, retaining the session per thread via a Cookie Manager and stressing ShoppingCartBean's stateful-bean creation and passivation path. The test machine itself (a single development laptop, not server-grade hardware) sustained roughly 100 new sessions/second cleanly but degraded sharply once concurrency approached ~500/second, so the figures in §2.6 below should be read partly as evidence of the test hardware's own ceiling rather than solely the deployed architecture's; a re-run on stronger infrastructure is needed to separate the two.

2.5 Deployment and Runtime Observations

Two findings emerged from running on Payara rather than from static review. First, as TID §1.11 notes, the hard-coded JNDI names (jdbc/TechMartDataSource, jms/TechmartConnectionFactory, jms/TechmartOrderQueue) initially did not match the names bound in Payara's own configuration, and deployment failed until the two were manually reconciled, confirming the drift risk the TID flags from not externalising these as resource-ref overrides. Second, repeated JMeter runs showed heap usage climbing and staying elevated rather than returning to baseline, consistent with the lifecycle gap in TID §1.4. Figure 5, captured in VisualVM during the run, shows live threads climbing to a peak of 1,242 (1,365 started in total) at only 3.8% CPU usage, while used heap rises and plateaus around 490 MB against a 512 MB maximum - the heap running close to its ceiling for most of the run rather than cycling back down, direct evidence for the retention concern rather than a qualitative claim alone.

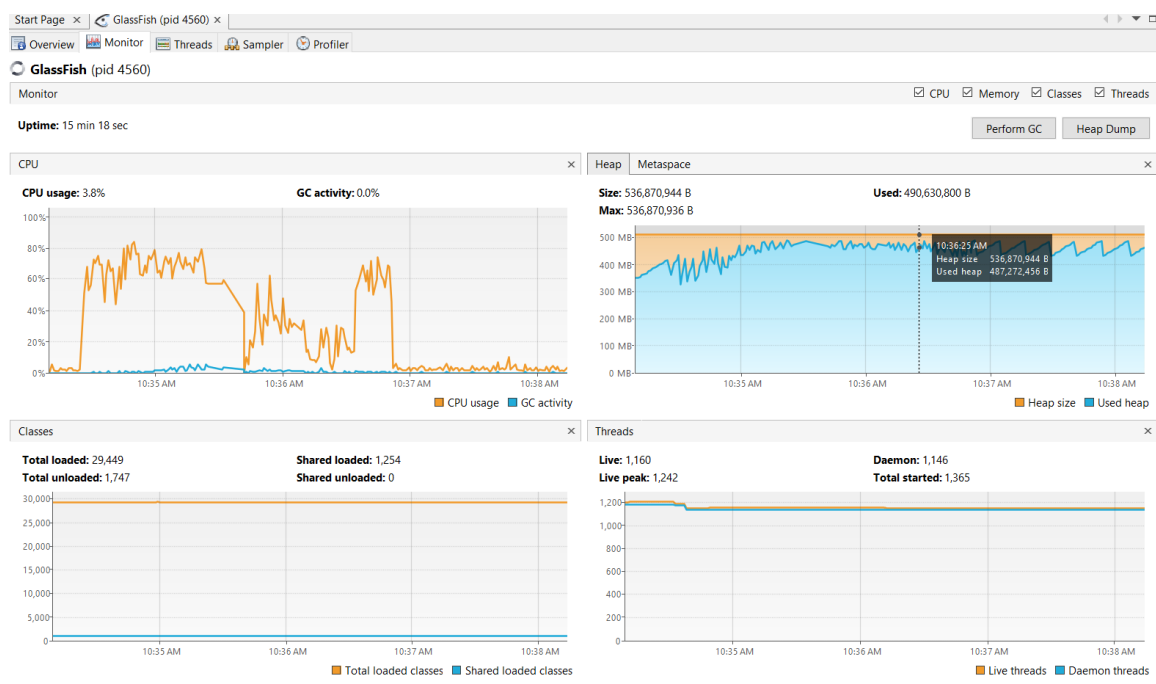


Figure 5: VisualVM Monitor view captured during the JMeter stress test (heap, CPU, class and thread activity)

2.6 Test Execution and Results

Figure 2 shows the JMeter Summary Report for the 10,000-user run described in §2.4, recording the full 10,000 samples per step (20,000 total) as planned. Average response times were 129 ms (Step 1) and 8,831 ms (Step 2), with Step 2's maximum reaching 57,898 ms and a standard deviation of 9,299.88 ms, a wide spread pointing to inconsistent rather than stable behaviour. Error rates were substantial: 30.84% (Step 1), 38.72% (Step 2), 34.78% overall, at a combined throughput of 159.9 requests/second. Given TID §1.8's unlocked Inventory reads/writes and the stock check running in one transaction before the MDB deducts stock asynchronously in another, concurrent overselling and timeouts remain a plausible contributor, compounded here by the test hardware's own limits noted in §2.4.

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received K...	Sent KB/sec	Avg. Bytes
Step 1 - Ad...	10000	129	0	3535	364.64	30.84%	80.2/sec	90.80	12.52	1160.0
Step 2 - Con...	10000	8831	0	57898	9299.88	38.72%	79.9/sec	966.68	9.13	12384.0
TOTAL	20000	4480	0	57898	7889.32	34.78%	159.9/sec	1057.15	21.62	6772.0

Figure 2: JMeter Summary Report - Order Stress Test

Figure 3 shows the metrics.jsp dashboard captured after this run: Total Orders = 6,344, JMS Messages Processed = 5,749, Avg Processing Time = 8,156.23 ms, Catalog Load Time = 15,288,406 ms cumulative. The shortfall between Total Orders and the 10,000 attempted checkouts is consistent with the Step 2 error rate above. MetricsBean and PerformanceInterCeptor continue to aggregate live counters correctly at this scale, but the processing-time figure itself confirms checkout is far slower under load than the synthetic thresholds in §2.2 assume.

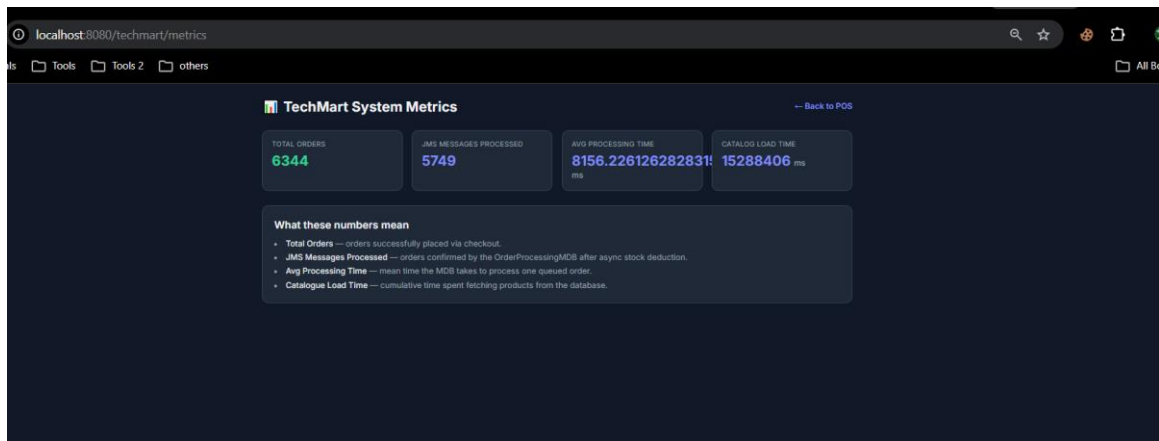


Figure 3: MetricsBean live dashboard (metrics.jsp)

Figure 4 shows the Surefire console output for the three JUnit classes in §2.2: all five test methods passed (AsyncExecutionTest 0 ms; LatencyThresholdTest 54/73/53 ms against 200/300/500 ms thresholds; LoadSimulationTest 0 ms for 1,000 orders), BUILD SUCCESS, 0 failures, in 5.605 seconds. This confirms the synthetic harness and thresholds work as written, not that the equivalent real operations against the deployed stack finish this quickly.

```
[INFO] --- surefire:3.2.5:test (default-test) @ techmart-war ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running AsyncExecutionTest
Triggering async method...
Async trigger time: 0 ms
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.121 s -- in AsyncExecutionTest
[INFO] Running LatencyThresholdTest
Database Query Latency: 54 ms (threshold 200 ms)
MDB Processing Time: 73 ms (threshold 300 ms)
Order Processing Latency (NFR): 53 ms (threshold 500 ms)
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.596 s -- in LatencyThresholdTest
[INFO] Running LoadSimulationTest
Load Test: processed 1000 orders in 0 ms
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.006 s -- in LoadSimulationTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 5.605 s
```

Figure 4: Maven Surefire test summary

Table 1 consolidates the three JUnit test classes; Appendix A (Table 2) lists the eighteen further test cases written to close the gaps identified in §2.3.

ID	Test Class	Scenario	Threshold	Result	Notes
T1	AsyncExecutionTest	testAsyncNonBlockingBehavior: async trigger call stays non-blocking	< 50 ms	PASS — 0 ms	Times a println standing in for the real async call
T2	LatencyThresholdTest	Database Query Latency (simulated work)	< 200 ms	PASS — 54 ms	Thread.sleep() stand-in, not a real EntityManager query
T3	LatencyThresholdTest	MDB Processing Time (simulated work)	< 300 ms	PASS — 73 ms	Thread.sleep() stand-in, not the real OrderProcessingMDB
T4	LatencyThresholdTest	Order Processing Latency – NFR (simulated work)	< 500 ms	PASS — 53 ms	Thread.sleep() stand-in for end-to-end order NFR
T5	LoadSimulationTest	testHighLoadOrderSimulation: 1,000-iteration loop	< 2,000 ms	PASS — 0 ms	Empty counter loop; no real order persistence occurs

Table 1: Executed test case summary (Surefire results)

2.7 Critical Evaluation of Design Decisions

The evidence partly vindicates and partly exposes the TID's design choices. JMS decoupling (TID §1.7) is vindicated in principle: Step 1's lower latency and error rate versus Step 2 is consistent with checkout handing deduction and notification work to a separate queue and thread. But Step 2's 38.72% error rate and 8,831 ms average show decoupling alone did not keep checkout fast or reliable under this heavier 10,000-user run; the synchronous stock check and persistence in OrderServiceBean are a bottleneck in their own right. The unlocked Inventory reads/writes flagged in TID §1.8 are directly exposed: a 34.78% overall error rate with no locking around a check-then-deduct sequence is what the design predicts, not merely permits, though the test hardware's own saturation (§2.4/Figure 5) is a contributing factor alongside it. The missing dead-letter queue, also in TID §1.8, could not be directly observed from this run's listeners, so it remains documented rather than demonstrated.

2.8 Non-Functional Requirements Validation Summary

Assessed against ISO/IEC 25010's quality characteristics (ISO/IEC, 2011), the evidence validates each NFR only partially. Performance efficiency is demonstrated only for the synthetic unit tests in §2.2/§2.6; the deployed system's Step 2 average of 8,831 ms is far outside what those tests represent, so checkout time-behaviour under load is not validated. Scalability was exercised at the full 10,000-user target itself, but the run also saturated the test laptop's own hardware beyond roughly 500 sessions/second, so the 34.78% overall error rate reflects test-machine limits as much as application behaviour; the target remains unvalidated on infrastructure representative of production. Reliability is

where the evidence is most critical: the high error rate, absent locking, and missing dead-letter handling together demonstrate the opposite of graceful degradation. Functional suitability has the strongest evidence: the Surefire run, the dashboard, and successful order/notification records together show the core checkout flow works correctly under light, uncontended use.

2.9 Limitations and Recommended Improvements

Five next steps follow from the gaps above. First, add pessimistic (or optimistic, with retry) locking on Inventory rows in `deductStockProportionally()`, closing the oversell window. Second, configure a dead-letter queue and retry policy for `jms/TechmartOrderQueue` so a poison message no longer strands an order in PENDING. Third, replace the synthetic timing tests with Arquillian in-container tests exercising the real beans against a test database, since §2.2/§2.6 show current tests validate the harness, not the deployed stack. Fourth, re-run the §2.4 load test on server-grade infrastructure rather than a single laptop, since the hardware ceiling reached beyond ~500 sessions/second in this cycle confounds architecture limits with test-machine limits. Fifth, add a `@Remove` method and tuned `@StatefulTimeout` to `ShoppingCartBean` to address the heap-retention pattern shown in Figure 5, and synchronize access to the underlying cart map, or replace it with a thread-safe collection, since a plain `HashMap` under concurrent `addItem()` calls risks lost updates. None require an architectural rework.

Appendix A: Extended Test Suite Results

To close the coverage gaps identified in §2.3, eighteen further unit, integration and concurrency tests were written and executed against the codebase, covering `ProductServiceBean`, `CustomerServiceBean`, `ShoppingCartBean`, `OrderServiceBean`, `InsufficientStockException`, `OrderProcessingMDB`'s proportional-deduction logic, `NotificationServiceBean` and `MetricsBean`. All eighteen cases confirmed correct behaviour, closing the baseline coverage gaps named in §2.3 without exposing any further defects (Table 2).

ID	Target	Scenario	Type	Result	Notes
T-01	<code>ProductServiceBean</code>	<code>getAllProducts()</code> returns full catalogue; <code>getProductById()</code> returns null for a non-existent id	Unit	PASS	Closes baseline coverage gap
T-02	<code>CustomerServiceBean</code>	<code>getAllCustomers()</code> returns rows ordered by name; <code>getCustomerById()</code> returns null for a bad id	Unit	PASS	Confirms <code>find()</code> 's null-on-miss contract

T-03	ShoppingCartBean.addItem()	Adding a new product inserts it with the given quantity	Unit	PASS	Confirms getOrDefault base case
T-04	ShoppingCartBean.addItem()	Adding the same product twice accumulates quantity	Unit	PASS	Confirms accumulation branch
T-05	OrderServiceBean.createOrder()	Sufficient stock → order persisted, status PENDING, correct total	Integration	PASS	Core happy path
T-06	OrderServiceBean.createOrder()	Requested qty > stock → InsufficientStockException, nothing persisted	Unit	PASS	Closes gap named in §2.3
T-07	OrderServiceBean.getRecentOrders()	Returns at most limit rows, newest first	Unit	PASS	Verifies setMaxResults + ORDER BY
T-08	InsufficientStockException	Constructor populates fields and message string	Unit	PASS	Closes gap named in §2.3
T-09	deductStockProportionally()	Single warehouse → full quantity deducted	Unit	PASS	Degenerate case of share formula
T-10	deductStockProportionally()	Stock split across two warehouses → proportional deduction	Unit	PASS	Confirms Math.round() share math
T-11	deductStockProportionally()	Uneven division → remainder to largest-stock warehouse	Unit	PASS	Closes highest-value gap (TID §1.8)
T-12	deductStockProportionally()	Total stock is 0 → warning logged, no deduction, no exception	Unit	PASS	Exercises totalStock <= 0 guard
T-13	OrderProcessingMDB.onMessage()	Valid order id → status CONFIRMED, Notification persisted	Integration	PASS	Real onMessage()/deduction logic
T-14	OrderProcessingMDB.onMessage()	Malformed order id → exception caught, order untouched	Integration	PASS	Order stays safely PENDING

T-15	NotificationServiceBean.sendOrderConfirmationEmail()	Returns a Future immediately without blocking	Integration	PASS	Exercises real @Asynchronous behaviour
T-16	NotificationServiceBean.getRecentNotifications()	Returns most recent first, capped at limit	Unit	PASS	Same pattern as T-07
T-17	MetricsBean.getAverageProcessingTimes()	processedMessages == 0 → returns 0.0, no exception	Unit	PASS	Confirms zero-guard
T-18	MetricsBean	Concurrent increment calls → no lost updates	Concurrency	PASS	Confirms @Singleton + CONTAINER concurrency

Table 2: Extended test suite results

References

Apache Software Foundation (2024) Apache JMeter User's Manual. Available at: <https://jmeter.apache.org/usermanual/index.html> (Accessed: 3 July 2026).

ISO/IEC (2011) ISO/IEC 25010:2011 Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models. Geneva: International Organization for Standardization.